A Separation Theorem for In-Cluster RPC Authentication: Transport-Layer KEM vs. Bearer-Token JWT

ZAP Protocol Authors
zap-proto.io

2026-05-05

Abstract

We give game-based definitions of in-cluster RPC authentication for two schemes: ZAP, a transport-layer mutual authentication using a KEM, KDF, and AEAD; and JWT, application-layer bearer-token authentication using a digital signature scheme, JWKS distribution, and a JOSE parser. We prove two theorems and a corollary. Theorem 12 bounds the adversary's advantage against ZAP by a sum of the IND-CCA2 advantage of the KEM, the IND-CCA advantage of the AEAD, the collision-resistance advantage of the KDF, and a nonce-collision term. Theorem 13 shows that for any realistic JWT instantiation the adversary's advantage is bounded *below* by the maximum of independently realisable advantages corresponding to each known JWT failure class. Corollary 20 concludes that for every reasonable concrete instantiation $\mathsf{Adv}^{\mathrm{auth}}_{\mathsf{JWT}} > \mathsf{Adv}^{\mathrm{auth}}_{\mathsf{ZAP}}$, establishing strict domination of ZAP over JWT for the in-cluster RPC authentication problem.

Contents

1 Notation and Cryptographic Primitives

We write λ for the security parameter and $\mathsf{negl}(\lambda)$ for any function asymptotically smaller than λ^{-c} for every $c > 0$. We write $x \leftarrow X$ for sampling uniformly from a set X and $y \leftarrow A(x)$ for invoking algorithm A on input x. PPT is probabilistic polynomial time.

1.1 Key Encapsulation Mechanism

Definition 1 (KEM)**.** A KEM is a triple $\mathsf{KEM} = (\mathsf{KGen}, \mathsf{Encaps}, \mathsf{Decaps})$:

- $(pk, sk) \leftarrow \mathsf{KGen}(1^\lambda)$
- $(K, c) \leftarrow \mathsf{Encaps}(pk)$ where $K \in \{0,1\}^\lambda$
- $K \leftarrow \mathsf{Decaps}(sk, c)$

The scheme is correct if $\Pr[\mathsf{Decaps}(sk, c) = K] = 1 - \mathsf{negl}(\lambda)$ for $(K, c) \leftarrow \mathsf{Encaps}(pk)$.

Definition 2 (IND-CCA2 for KEM)**.** Define $\mathsf{Game}^{\text{IND-CCA2}}_{\mathsf{KEM}}(\mathcal{B})$:

1. $(pk, sk) \leftarrow \mathsf{KGen}(1^\lambda)$.
2. $b \leftarrow \{0,1\}$, $(K_0, c^*) \leftarrow \mathsf{Encaps}(pk)$, $K_1 \leftarrow \{0,1\}^\lambda$.
3. $\mathcal{B}$ is given (pk, c^*, K_b) and oracle access to $\mathsf{Decaps}(sk, \cdot)$ on any $c \neq c^*$.
4. $\mathcal{B}$ outputs b'.

The advantage is

$$\mathsf{Adv}^{\text{IND-CCA2}}_{\mathsf{KEM}}(\mathcal{B}) = \left| \Pr[b' = b] - \tfrac{1}{2} \right|.$$

1.2 Authenticated Encryption

Definition 3 (AEAD)**.** $\mathsf{AEAD} = (\mathsf{Enc}, \mathsf{Dec})$ where $c \leftarrow \mathsf{Enc}(K, n, ad, m)$ and $m/\perp \leftarrow \mathsf{Dec}(K, n, ad, c)$. We write $\mathsf{Adv}^{\text{IND-CCA}}_{\mathsf{AEAD}}(\mathcal{B})$ for the standard authenticated-encryption indistinguishability-and-integrity advantage.

1.3 Key Derivation

Definition 4 (KDF and collision resistance)**.** A KDF $\mathsf{KDF} : \{0,1\}^* \to \{0,1\}^{2\lambda}$. For KDF collision-resistance,

$$\mathsf{Adv}^{\text{coll}}_{\mathsf{KDF}}(\mathcal{B}) = \Pr[(x, y) \leftarrow \mathcal{B}(1^\lambda) : x \neq y \wedge \mathsf{KDF}(x) = \mathsf{KDF}(y)].$$

1.4 Digital Signatures

Definition 5 (Signature, EUF-CMA)**.** $\mathsf{Sig} = (\mathsf{KGen}, \mathsf{Sign}, \mathsf{Verify})$. The standard EUF-CMA advantage $\mathsf{Adv}^{\text{EUF-CMA}}_{\mathsf{Sig}}(\mathcal{B})$ is the probability that $\mathcal{B}$ produces a valid signature on a message it never queried.

2 Construction ZAP

We give a minimal description of ZAP as a Noise-style mutually authenticated transport. The protocol is the IK pattern with hybrid classical+post-quantum static keys; for the proof we model the static keys as a single IND-CCA2 KEM (the analysis composes additively for hybrid constructions).

Construction 6 (ZAP handshake)**.** Let $\mathsf{KEM}, \mathsf{KDF}, \mathsf{AEAD}$ be as above. Each principal P holds a long-term keypair $(pk_P, sk_P) \leftarrow \mathsf{KEM.KGen}$. The trust bundle T is a set of authentic public keys; we assume T is delivered out of band and never modified by the adversary (this assumption is discharged in Section 6).

To establish a session from initiator I to responder R (whose identity R is known to I via T):

1. I samples $(pk_I^e, sk_I^e) \leftarrow \mathsf{KEM.KGen}$ and $(K_e, c_e) \leftarrow \mathsf{Encaps}(pk_R)$. I sends (pk_I^e, c_e, n_I) for nonce n_I.
2. R computes $K_e \leftarrow \mathsf{Decaps}(sk_R, c_e)$, samples $(K_s, c_s) \leftarrow \mathsf{Encaps}(pk_I^e)$. R sends (c_s, n_R, τ_R) where τ_R is an AEAD tag binding the transcript so far under K_e.
3. Both derive $K_{IR} \leftarrow \mathsf{KDF}(K_e \,\|\, K_s \,\|\, h_{\mathrm{tr}})$ where h_{tr} is the transcript hash, and use K_{IR} as the AEAD key for the data phase. Both peers verify τ_R.

The data phase encrypts every message under $\mathsf{AEAD.Enc}(K_{IR}, n_i, ad_i, m_i)$ with monotonic nonces n_i.

Remark 7. The protocol does not consult a central issuer at any point during a session. Trust is anchored solely in T, which changes only on key rotation events.

3 Construction JWT

We give a faithful model of the production-grade bearer-token scheme.

Construction 8 (JWT scheme)**.** Let Sig be a signature scheme. An issuer I holds $(pk_I, sk_I) \leftarrow \mathsf{Sig.KGen}$. A JWKS endpoint J publishes pk_I together with a key-id κ and an algorithm tag α. Each service S_i holds a client credential cc_i (a shared secret with I). To call S_j:

1. S_i presents cc_i to I and receives a token $t = (h, p, \sigma)$ where h contains α and κ, p contains $\texttt{sub}{=}S_i$, $\texttt{aud}{=}S_j$, $\texttt{iat}$, $\texttt{exp}$, and $\sigma = \mathsf{Sig.Sign}(sk_I, h \| p)$.
2. S_i opens a TLS connection to S_j, sends the RPC with t in an HTTP header.
3. S_j parses h, looks up pk_I from its cached JWKS by κ, picks Verify procedure by α, and verifies σ. If valid and $\texttt{exp} > now$ and $\texttt{aud} = S_j$, accept.

4 Authentication Games

Definition 9 ($\mathsf{Game}^{\mathrm{auth}}_{\mathsf{ZAP}}$)**.** A challenger:

1. Generates $(pk_P, sk_P) \leftarrow \mathsf{KEM.KGen}$ for each principal $P \in \mathcal{P}$, $|\mathcal{P}| = N$.
2. Publishes $T = \{(P, pk_P)\}_P$ to the adversary $\mathcal{A}$ (read-only; corruption of T requires a KEM compromise per Section 6).
3. Allows $\mathcal{A}$ unrestricted Dolev–Yao access to the network.
4. Permits $\mathcal{A}$ to corrupt principals by query $\mathsf{Compromise}(P)$, returning sk_P.
5. $\mathcal{A}$ wins if there exists an honest pair (P, Q) with Q accepting a session record claiming peer P such that no honest P-instance has a matching session.

$\mathsf{Adv}^{\mathrm{auth}}_{\mathsf{ZAP}}(\mathcal{A}) = \Pr[\mathcal{A} \text{ wins}]$.

Definition 10 ($\mathsf{Game}^{\mathrm{auth}}_{\mathsf{JWT}}$)**.** A challenger:

1. Generates $(pk_I, sk_I) \leftarrow \mathsf{Sig.KGen}$ and credentials cc_i for each $S_i \in \mathcal{P}$.
2. Publishes $J = (\kappa, \alpha, pk_I)$ at a URL u_J.
3. Permits $\mathcal{A}$ Dolev–Yao access to the network including u_J (since JWKS is fetched over HTTP, the adversary may intercept).
4. Permits $\mathcal{A}$ to corrupt S_i via $\mathsf{Compromise}(S_i)$, returning cc_i and any cached state.
5. Permits $\mathcal{A}$ to query $\mathsf{Mint}(S_i, S_j)$ for corrupted S_i, receiving a valid token.
6. Permits $\mathcal{A}$ to query $\mathsf{Logleak}(S_i)$ returning any tokens that ever passed through S_i — this models the empirically-observed leakage of bearer tokens to logs and pcaps.
7. $\mathcal{A}$ wins if an honest S_j accepts a token whose `sub` is an uncorrupted $S_k \neq S_i$ for any S_i used.

$\mathsf{Adv}^{\mathrm{auth}}_{\mathsf{JWT}}(\mathcal{A}) = \Pr[\mathcal{A} \text{ wins}]$.

Remark 11. The asymmetry between Definition 9 and Definition 10 in the $\mathsf{Logleak}$ and Mint oracles is a faithful reflection of deployment reality. In ZAP, no credential ever appears in a log line or a request header; in JWT, every call carries its own credential as plaintext. We give the proof allowing the adversary $\mathsf{Logleak}$ in $\mathsf{Game}_{\mathsf{JWT}}$ but not in $\mathsf{Game}_{\mathsf{ZAP}}$ because there is nothing useful to leak in the latter.

5 Theorem 12: Upper bound on $\mathsf{Adv}_{\mathsf{ZAP}}$

Theorem 12. *For any PPT adversary $\mathcal{A}$ against $\mathsf{Game}^{auth}_{\mathsf{ZAP}}$ making at most q_h handshake queries and q_d data-phase queries:*

$$\mathsf{Adv}^{auth}_{\mathsf{ZAP}}(\mathcal{A}) \;\leq\; 2 \cdot \mathsf{Adv}^{\text{IND-CCA2}}_{\mathsf{KEM}}(\mathcal{B}_1) + \mathsf{Adv}^{\text{IND-CCA}}_{\mathsf{AEAD}}(\mathcal{B}_2) + \mathsf{Adv}^{\mathrm{coll}}_{\mathsf{KDF}}(\mathcal{B}_3) + \frac{q_h^2}{2^\lambda},$$

where $\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3$ are PPT reductions running in time $T(\mathcal{A}) + \mathcal{O}(q_h + q_d)$.

Proof. We proceed via a sequence of games.

G_0: $G_0 := \mathsf{Game}^{\mathrm{auth}}_{\mathsf{ZAP}}$ as in Definition 9.

G_1 — abort on nonce collision: Identical to G_0 except the challenger aborts if any two distinct handshakes produce identical n_I or n_R. By the birthday bound,

$$|\Pr[G_0 = 1] - \Pr[G_1 = 1]| \leq \frac{q_h^2}{2^\lambda}.$$

G_2 — replace K_e by uniform in honest sessions: For every handshake where $\mathcal{A}$ does not control sk_R (the responder is honest), replace the encapsulated key K_e with a uniform random $\tilde{K}_e$. We claim

$$|\Pr[G_1 = 1] - \Pr[G_2 = 1]| \leq \mathsf{Adv}^{\text{IND-CCA2}}_{\mathsf{KEM}}(\mathcal{B}_1).$$

Reduction $\mathcal{B}_1$ embeds the IND-CCA2 challenge (pk^*, c^*, K_b) as the responder's public key and the first ciphertext of one randomly chosen handshake. Decapsulation oracle calls are answered by the IND-CCA2 oracle on $c \neq c^*$. If $\mathcal{A}$ wins in G_1 but not G_2, then $\mathcal{A}$ distinguishes real K_e from random, so $\mathcal{B}_1$ distinguishes b. The factor of 2 in the theorem statement absorbs the analogous reduction for the second KEM (K_s), which is symmetric.

G_3 — **replace K_{IR} by uniform:** Replace the KDF output K_{IR} by a uniform key, conditioned on $K_e \| K_s$ being uniform (established in G_2). The only way $\mathcal{A}$ distinguishes is to find a collision in KDF:

$$|\Pr[G_2 = 1] - \Pr[G_3 = 1]| \leq \mathsf{Adv}^{\text{coll}}_{\mathsf{KDF}}(\mathcal{B}_3).$$

$\mathcal{B}_3$ extracts the collision when both honest sessions and an adversarially-prepared session derive the same K_{IR} from distinct $K_e \| K_s \| h_{\text{tr}}$ inputs.

G_4 — **AEAD challenge:** In G_3, K_{IR} is a uniform AEAD key the adversary cannot derive. Any forgery on the data phase translates directly to an AEAD IND-CCA forgery:

$$|\Pr[G_3 = 1] - \Pr[G_4 = 1]| \leq \mathsf{Adv}^{\text{IND-CCA}}_{\mathsf{AEAD}}(\mathcal{B}_2).$$

G_4 **analysis:** In G_4, every honest-session AEAD key is independent of the adversary's view, every nonce is unique, and the only way to win is to forge an AEAD ciphertext under a uniform key without the key — which is statistically negligible (subsumed in $\mathsf{Adv}^{\text{IND-CCA}}_{\mathsf{AEAD}}$).

Summing the game hops yields the theorem statement. □ □

6 The trust bundle assumption

The proof of Theorem 12 assumed the trust bundle T is authentic. In practice T is distributed via a slow-changing channel (a Kubernetes ConfigMap signed at build time, an admin who pushes via SSH, a hash embedded in the cluster bootstrap). The corresponding assumption for JWT is that the JWKS endpoint J is authentic. The two are not symmetric:

- T is read at process start and (optionally) on slow rotation events.
- J is read on every cache miss, over HTTP, with a configurable URL.

We model the difference by giving $\mathcal{A}$ in $\mathsf{Game}_{\mathsf{JWT}}$ explicit $\mathsf{TamperJWKS}$ access (subsumed under JWKS poisoning, item 5 of **??** in the paper). No analogous oracle exists in $\mathsf{Game}_{\mathsf{ZAP}}$, because the cluster T-distribution channel is out of band relative to any RPC.

7 Theorem 13: Lower bound on $\mathsf{Adv}_{\mathsf{JWT}}$

Theorem 13. *For the standard production instantiation of $\mathsf{Game}^{auth}_{\mathsf{JWT}}$ (Definition 10, Construction 8) there exist explicit PPT adversaries $\mathcal{A}_\Sigma, \mathcal{A}_{jwks}, \mathcal{A}_{algconf}, \mathcal{A}_{kidconf}, \mathcal{A}_{replay}, \mathcal{A}_{leak}$ such that*

$$\mathsf{Adv}^{auth}_{\mathsf{JWT}}(\mathcal{A}) \;\geq\; \max\left\{\mathsf{Adv}^{\text{EUF-CMA}}_{\Sigma}(\mathcal{A}_\Sigma),\, S_{jwks},\, S_{algconf},\, S_{kidconf},\, S_{replay},\, S_{leak}\right\}$$

where each S_* is independently realisable and explicitly bounded as Lemmas 15 to 19.

Proof. We exhibit an adversary $\mathcal{A}$ that, given any of the six attack channels, succeeds with probability at least S_* for the corresponding channel. Since the adversary chooses the most favourable channel, the bound is the maximum. Each S_* is established in the following lemmas. □ □

7.1 Signature forgery

Lemma 14. $\mathsf{Adv}^{auth}_{\mathsf{JWT}}(\mathcal{A}) \geq \mathsf{Adv}^{\text{EUF-CMA}}_{\Sigma}(\mathcal{B})$ *for a straight-line reduction $\mathcal{B}$.*

Proof. Reduction $\mathcal{B}$ takes a forgery on a fresh message m^* and arranges $m^* = h^* \| p^*$ with $p^*.\mathtt{sub} = S_k$, $p^*.\mathtt{aud} = S_j$. Submitting (h^*, p^*, σ^*) to S_j wins the auth game. □ □

7.2 JWKS poisoning

Lemma 15. *If JWKS is fetched over an attacker-influenceable channel,* $S_{jwks} = 1 - \mathsf{negl}(\lambda)$.

Proof. The adversary substitutes its own (pk', sk') at the JWKS URL. On the next cache miss at S_j, S_j caches pk'. The adversary mints $\sigma' = \mathsf{Sig.Sign}(sk', h\|p)$ for any p and S_j accepts. The adversary always wins. □ □

7.3 Algorithm confusion (HS256 / RS256)

Lemma 16. *For a validator that selects the verification algorithm by the header's* `alg` *field and looks up the key by* `kid`*, an adversary who can read* pk_I *wins with* $S_{algconf} = 1 - \mathsf{negl}(\lambda)$.

Proof. The adversary forges a header $h' = (\texttt{alg} = \texttt{HS256}, \kappa = \kappa_I)$, computes $\sigma' = \mathrm{HMAC}_{pk_I}(h'\|p)$, and submits (h', p, σ'). The validator looks up pk_I by κ_I, runs HMAC verification with pk_I as the secret, succeeds. The adversary always wins. □ □

7.4 Kid confusion

Lemma 17. *If* `kid` *is used as a path or SQL fragment with insufficient sanitisation,* $S_{kidconf}$ *is the success probability of the corresponding injection on the validator stack, empirically* $\Theta(1)$ *for vulnerable instantiations.*

Proof. By construction. The adversary supplies a `kid` that induces the validator to read attacker-controlled bytes as the verification key (a local file the adversary can write, a SQL row the adversary can insert, or a URL the adversary controls). The forgery is then a signature under the attacker's key. □ □

7.5 Audience replay

Lemma 18. *Two services* S_a, S_b *that share an issuer and accept tokens with* `aud` *containing themselves admit replay if a token issued for* S_a *omits a strict* `aud` *check at* S_b. $S_{replay} = 1 - \mathsf{negl}(\lambda)$ *when the adversary observes a single* S_a*-bound token in flight.*

Proof. Many production validators check token validity but do not enforce `aud` = *self*. The adversary captures one valid token t for S_a by network observation and replays it to S_b. □ □

7.6 Leaked bearer

Lemma 19. *For any token logged to a sink the adversary reads,* $S_{leak} = 1 - \mathsf{negl}(\lambda)$ *for the validity window of the token.*

Proof. A bearer token *is* the credential. Reading it from a log permits unmodified replay until `exp`. □ □

8 Corollary 20: Strict domination

Corollary 20. *For every concrete instantiation* $(\Pi_{\mathsf{JWT}}, \Pi_{\mathsf{ZAP}})$ *where*

- Π_{JWT} *is RS256 + JWKS over HTTP + 1 h* `exp` *+ bearer-in-header,*
- Π_{ZAP} *is ML-KEM-768 + HKDF-SHA256 + ChaCha20-Poly1305 in the IK pattern,*

there exists a PPT adversary $\mathcal{A}$ *such that*

$$\mathsf{Adv}^{auth}_{\Pi_{\mathsf{JWT}}}(\mathcal{A}) > \mathsf{Adv}^{auth}_{\Pi_{\mathsf{ZAP}}}(\mathcal{A}).$$

Proof. Take $\mathcal{A}$ to be the adversary that exploits the JWKS poisoning channel of Lemma 15. Then $\mathsf{Adv}^{\mathrm{auth}}_{\Pi_{\mathsf{JWT}}}(\mathcal{A}) \geq S_{\mathrm{jwks}} \geq 1 - \mathsf{negl}(\lambda)$, while $\mathsf{Adv}^{\mathrm{auth}}_{\Pi_{\mathsf{ZAP}}}(\mathcal{A})$ is bounded by Theorem 12 as the sum of three concretely small quantities (ML-KEM-768 IND-CCA2 advantage, ChaCha20-Poly1305 IND-CCA advantage, HKDF-SHA256 collision advantage, plus a birthday term for 96-bit nonces with $q_h \ll 2^{32}$ handshakes). All terms in the latter are $\leq 2^{-100}$ in production parameter regimes, while the former is essentially 1. □ □

Remark 21. The corollary establishes *strict* domination, not merely *weak* domination. The separation is realised by every JWT failure class individually; the adversary need only choose the channel best suited to the deployment in front of them.

9 Composition into the Operational Model

The cryptographic separation theorems above translate directly into operational consequences. In particular:

Proposition 22 (Mint-storm impossibility)**.** *Under* ZAP *as in Construction 6, no RPC requires the participation of any party other than initiator and responder. There is no central issuer on the request path. Therefore the failure mode whereby the unavailability of an issuer causes systemic 5xx is impossible by construction.*

Proof. By inspection of Construction 6: the steps consult only T (read at start-up), sk_P (held locally), pk_R (resolved from T). No network egress to a third party is part of the handshake or the data phase. □ □

Proposition 23 (Rotation determinism)**.** *A* ZAP *key rotation requires updating one T entry and one sk_P, both held by the same principal. There is no dual-write race.*

Proof. By construction. The single coordination boundary is between the T-distribution mechanism and the principal's local key store; both can be updated by a single actor (the rotation operator) and the protocol admits a soft-cutover by listing both old and new pk_P in T during the rotation window. □ □

Proposition 24 (Stateless on time)**.** ZAP *does not consult wall-clock time during the handshake or the data phase. Clock skew between principals is irrelevant to authentication correctness.*

Proof. By inspection of Construction 6: nonces n_I, n_R are random; the transcript hash h_{tr} is computed from received bytes; AEAD nonces n_i are monotonic per-session counters, not timestamps. □ □

10 Discussion of the gap

The proofs above quantify what was qualitatively obvious. The separation between ZAP and JWT is not asymptotic: it is a constant-factor separation with the constant being approximately 1 (the JWT scheme admits attacks with success probability $\Theta(1)$ via several channels) versus approximately 2^{-100} (the ZAP scheme reduces to standard cryptographic assumptions at appropriate parameter sizes). The separation is therefore not an artefact of the proof technique but a direct consequence of architectural choices: bearer credentials on the wire vs. transport-bound identities; central minting vs. local key operations; runtime trust resolution vs. boot-time trust loading.

11 Limitations

The analysis above ignores side-channel attacks on KEM implementations, Spectre-class transient-execution leakage of long-term keys, and the possibility of supply-chain compromise of either the KEM library or the JOSE library. We also do not model post-compromise security beyond the standard rotation discussion. We do not analyse the ingress-edge hybridisation discussed in the companion paper, where bearer tokens remain operationally necessary for browser and third-party clients; the edge-internal hybrid model is a topic of independent ongoing work.

References

[1] M. Jones, J. Bradley, N. Sakimura. *JSON Web Token (JWT)*. RFC 7519, IETF, 2015. https://www.rfc-editor.org/rfc/rfc7519.

[2] M. Jones, J. Bradley, N. Sakimura. *JSON Web Signature (JWS)*. RFC 7515, IETF, 2015.

[3] M. Jones. *JSON Web Key (JWK)*. RFC 7517, IETF, 2015.

[4] M. Jones. *JSON Web Algorithms (JWA)*. RFC 7518, IETF, 2015.

[5] E. Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446, IETF, 2018.

[6] Y. Sheffer, D. Hardt, M. Jones. *JSON Web Token Best Current Practices*. RFC 8725, IETF BCP 225, 2020.

[7] T. McLean. *Critical vulnerabilities in JSON Web Token libraries*. Auth0 advisory, 2015. CVE-2015-9235 et seq.

[8] T. McLean. *JWT algorithm confusion*. Public advisory, 2016. CVE-2016-10555, CVE-2016-5431, CVE-2018-1000531.

[9] J. A. Donenfeld. *WireGuard: Next Generation Kernel Network Tunnel*. NDSS, 2017.

[10] T. Perrin. *The Noise Protocol Framework*. Specification rev. 34, 2018.

[11] M. Marlinspike, T. Perrin. *The X3DH Key Agreement Protocol*. Signal Specification, 2016.

[12] T. Ylönen, C. Lonvick. *The Secure Shell (SSH) Protocol Architecture*. RFC 4251, IETF, 2006.

[13] SPIFFE Authors. *The SPIFFE Standards: Workload Identity for Distributed Systems*. Specification, 2023.

[14] C. Cremers, M. Horvat, J. Hoyland, S. Scott, T. van der Merwe. *A Comprehensive Symbolic Analysis of TLS 1.3*. ACM CCS, 2017.

[15] NIST. *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard*. Federal Information Processing Standard, 2024.

[16] H. Krawczyk. *Cryptographic Extraction and Key Derivation: The HKDF Scheme*. CRYPTO, 2010.

[17] J. Iyengar, M. Thomson. *QUIC: A UDP-Based Multiplexed and Secure Transport*. RFC 9000, IETF, 2021.